

Task 2 - Video Submission

Customer International Payments Portal

Group Member Name:	Student Number:
Tshwarelo Lephoto	ST10535591
Gcina Nkosi	ST10103441
Lebogang Padi	ST10347256
Jabulani Kubeka	ST10504381
Mbulelo Hlobo	ST10506120

Links:

YouTube Demo Video: https://youtu.be/xvtoD_Yjslg

GitHub Repository: <https://github.com/CodeByRelo/InternationalPaymentsAPI.git>

Description:

This submission demonstrates a secure **Customer International Payments Portal** developed as part of Task 2. The system allows users to register, authenticate and simulate international payments while implementing strong security practices across all layers of the application.

The solution was built using the following technology stack:

- **Frontend:** React (JavaScript)
 - **Backend:** ASP.NET Core Web API
 - **Database:** SQL Server
 - **Authentication:** JSON Web Tokens (JWT)
 - **Hosting:**
 - Frontend hosted on Netlify (HTTPS enabled)
 - Backend hosted on Microsoft Azure (HTTPS enabled)
 - **CI/CD Pipeline:** GitHub Actions
-

Security Implementation Overview

The system was designed with a strong focus on protecting sensitive financial data. The following security measures were implemented and demonstrated in the video:

1. Password Security

- User passwords are **hashed and salted** using ASP.NET Identity's PasswordHasher.
- Plain-text passwords are never stored in the database.

- The video includes a demonstration of hashed passwords stored in SQL Server.

2. Input Validation (Whitelisting)

- All user inputs are validated using **Data Annotations and Regular Expressions (RegEx)**.
- Invalid inputs (such as incorrect ID numbers, weak passwords, invalid account numbers) are rejected.
- Validation errors are displayed clearly in the UI under each input field.

3. Authentication and Authorization

- Secure login is implemented using **JWT (JSON Web Tokens)**.
- Tokens include user claims such as role and account number.
- Role-based access control ensures only authorized users can access specific features.

4. Securing Data in Transit

- The application is hosted over **HTTPS (TLS encryption)** via Azure.
- **HSTS (HTTP Strict Transport Security)** is enforced to prevent downgrade attacks.
- All communication between client and server is encrypted.

5. Protection Against Common Web Attacks

Layers of protection included in the system:

- **Brute Force Attacks:** Rate limiting applied on login attempts
- **Cross-Site Scripting (XSS):** Input sanitisation, output encoding and Content Security Policy
- **SQL Injection:** Use of Entity Framework with parameterized queries
- **Clickjacking:** X-Frame-Options headers set to DENY
- **Man-in-the-Middle Attacks:** Enforced HTTPS and secure headers
- **Session Hijacking:** Secure token-based authentication and controlled session handling

6. Security Headers

Middleware applies headers such as:

- X-Frame-Options
- X-Content-Type-Options
- X-XSS-Protection
- Content-Security-Policy
- Strict-Transport-Security

DevSecOps Pipeline

A **CI/CD pipeline** was implemented using GitHub Actions.

The pipeline automatically runs on every push and performs:

- Dependency restoration
- Project build
- Automated testing
- Security checks for vulnerable packages

This ensures code quality, early vulnerability detection, and secure deployment practices.

Video Demonstration Overview

The submitted video demonstrates the following functionalities:

1. System overview and architecture
 2. User registration with input validation and password security
 3. User login and JWT-based authentication
 4. Customer dashboard functionality
 5. International payment creation and submission
 6. Display of validation errors and secure handling of input
 7. HTTPS secure connection
 8. Explanation of implemented security measures
 9. Demonstration of the DevSecOps pipeline
-

AI Disclosure

AI tools such as ChatGPT and Copilot were used to assist with development.

All generated code was reviewed and adapted to ensure it follows secure coding practices.

Conclusion

This project demonstrates a secure, functional, and well-structured customer payments system. By integrating authentication, validation, encryption and automated security processes, the system ensures the **confidentiality, integrity, and availability** of user and transaction data.